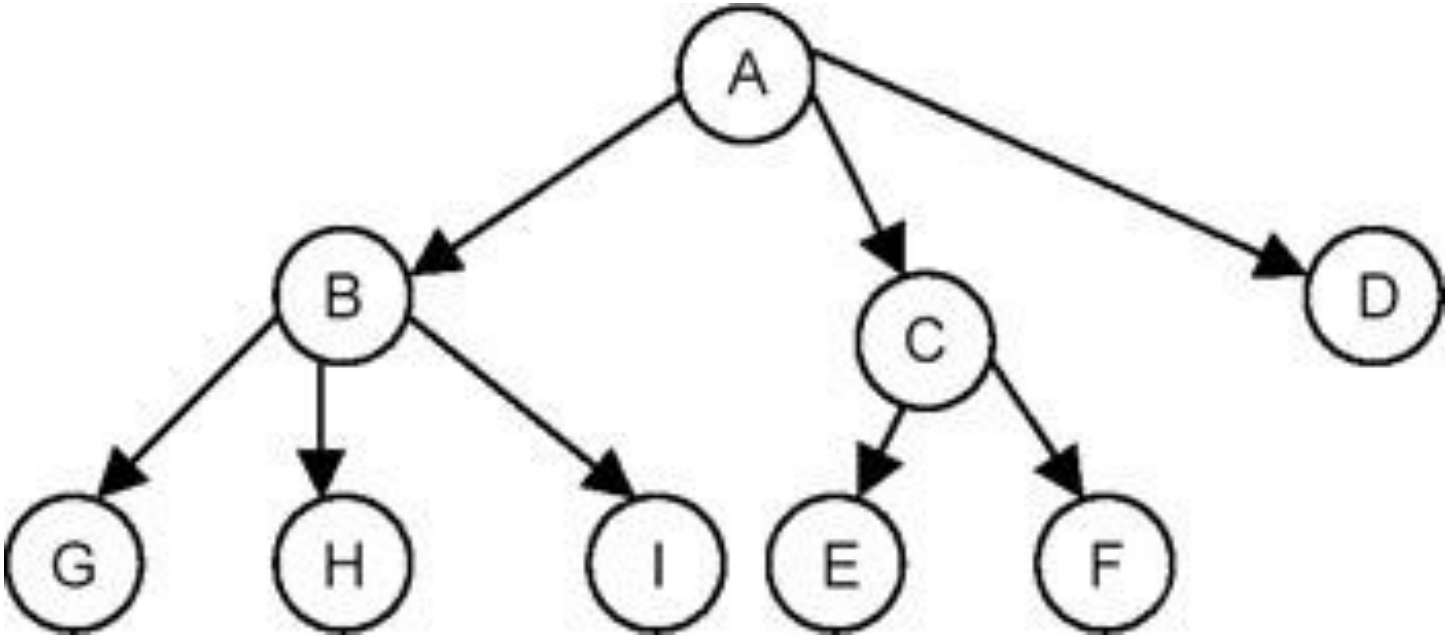
CHƯƠNG 5

CẤU TRÚC CÂY

Cây (trees)

- **Là một tập hợp các phần tử (nút) mang dữ liệu cùng kiểu, có trang bị mối quan hệ phân cấp "cha-con" thỏa mãn:**
  - Mỗi nút có thể có k nút con, $k \in N$
  - Trừ nút gốc không có cha, các nút còn lại có duy nhất một cha
  - Nếu B là con của A thì A là cha của B, và ngược lại

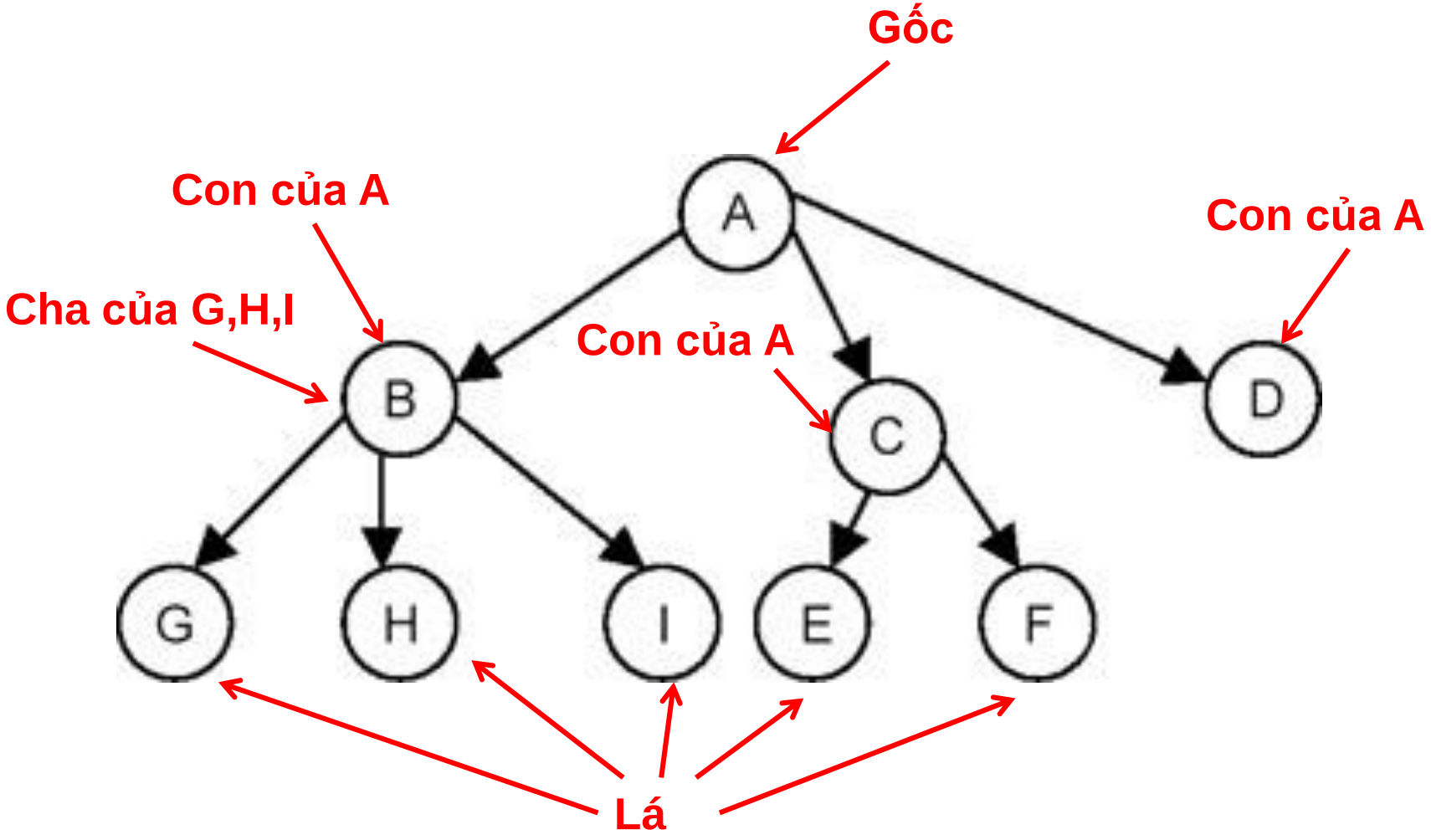
Cây (trees)



Cây (trees)

- **Nút gốc** : là nút không có nút cha.
- **Nút lá** : là nút không có con.
- **Nút cành** : là nút có ít nhất 1 con và không phải là gốc
- **Bậc (cấp) của một nút** : là số nút con của nút đó
- **Bậc (cấp) của một cây** : là bậc lớn nhất của các nút trong cây (số con tối đa của một nút thuộc cây).
  - Cây có bậc n thì gọi là cây n-phân.

Cây (trees)



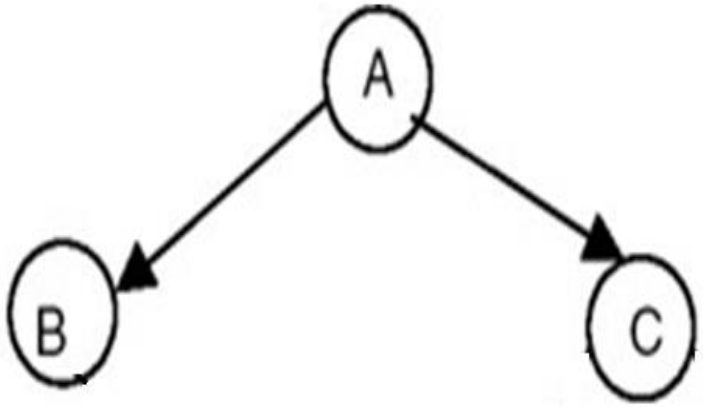
Cây (trees)

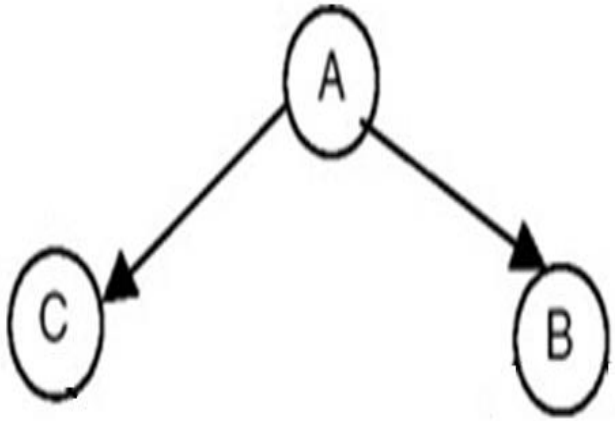
- **Mức của một nút :**
  - Mức (gốc (T_0)) = 1.
  - Gọi T_1, T_2, T_3, ... , T_n là các cây con của T_0 .

  Mức (T_1) = Mức (T_2) = ... = Mức (T_n) = Mức (T_0) + 1.
- **Chiều cao của cây là mức lớn nhất của các nút trên cây.**
- **Cây rỗng có chiều cao bằng 0.**

Cây (trees)

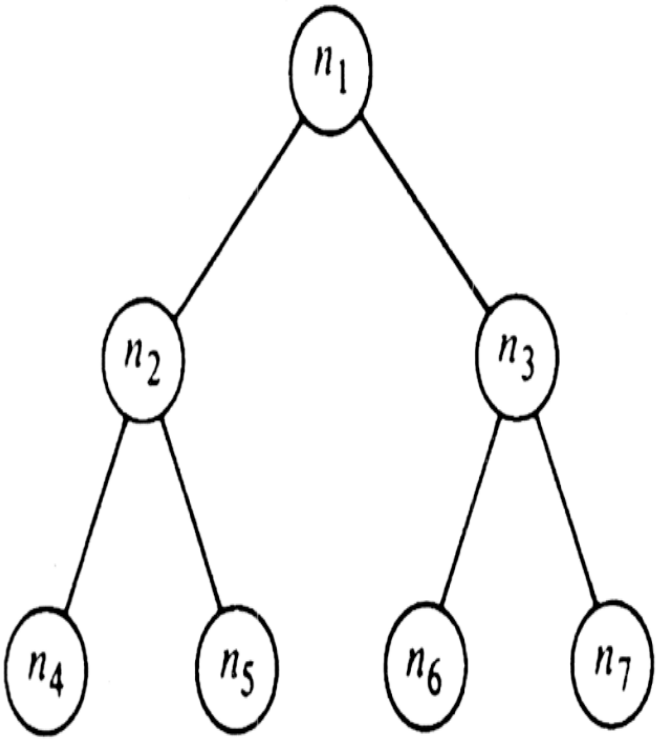
Cây có thứ tự là cây coi trọng thứ tự các nút con, ngược lại là cây không có thứ tự





Cây nhị phân (binary trees)

- Là cây có thứ tự và mỗi nút có tối đa 2 con (trái và phải)
- Số nút tối đa ở mức i là 2^{i-1} ,$i \geq 1$
- Số nút tối đa trên cây khác rỗng có chiều cao h là 2^h-1



Cây nhị phân (binary trees)

- Các cây nhị phân đặc biệt:
  - **Cây suy biến**: mỗi nút có tối đa một nút con
  - **Cây nhị phân hoàn chỉnh**: ngoại trừ mức kế cuối cùng, tất cả các nút ở các mức còn lại đều có 2 nút con. Riêng mức cuối cùng được lấp đầy từ trái sang phải.
  - **Cây nhị phân đầy đủ**: ngoại trừ nút lá, tất cả các nút còn lại đều có 2 nút con

Lưu trữ kế tiếp cây nhị phân

- Giả sử cây có chiều cao h
- Dùng mảng T có $n \geq (2^h-1)$ phần tử để lưu trữ:
  - Lưu theo thứ tự ưu tiên từng mức
  - Mỗi mức có đủ 2^{i-1} phần tử, nút khuyết tương ứng với phần tử trống

Lúc đó:

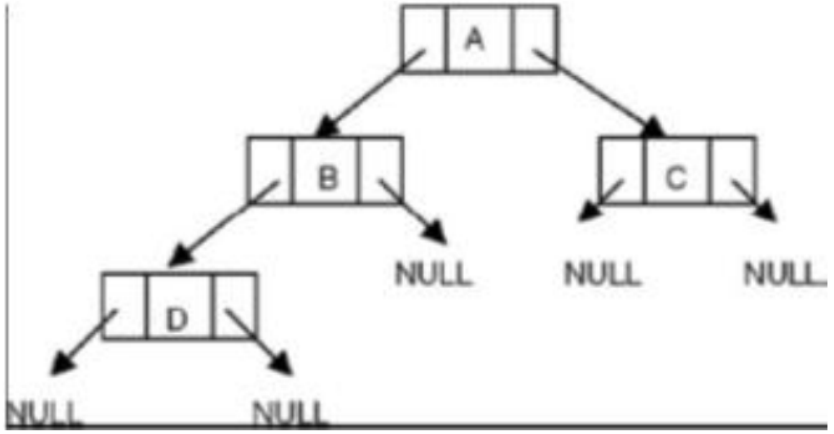
- T[1] là nút gốc
- T[i] có hai nút con là T[2i] và T[2i+1]
- T[i] có nút cha là T[i div 2]

Cách lưu này gây lãng phí bộ nhớ trong trường hợp cây không hoàn chỉnh!

Lưu trữ móc nối cây nhị phân

- Sử dụng một biến động để lưu trữ các thông tin của một nút:
  - Thông tin lưu trữ tại nút
  - Địa chỉ nút gốc của cây con trái trong bộ nhớ
  - Địa chỉ nút gốc của cây con phải trong bộ nhớ
- Khai báo cấu trúc cây nhị phân:

```
struct TNode
{
        DataType data;
        TNode   *pLeft, *pRight;
};
typedef TNode* Tree;
```



- Để quản lý cây nhị phân chỉ cần quản lý địa chỉ nút gốc:

```
Tree root;
```

DUYỆT CÂY NHỊ PHÂN

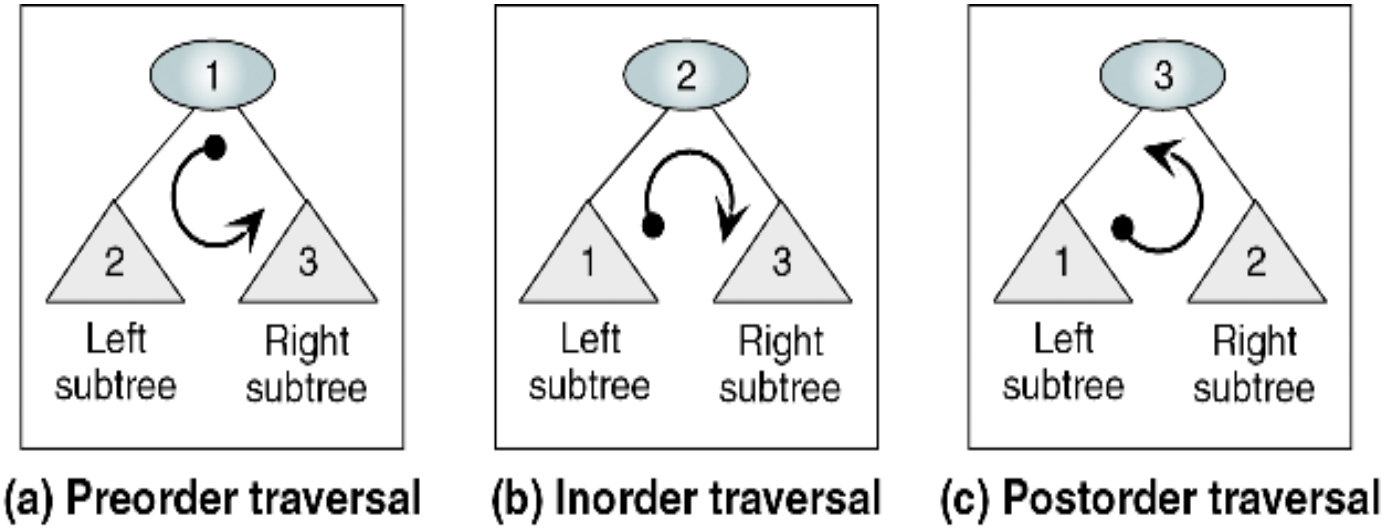
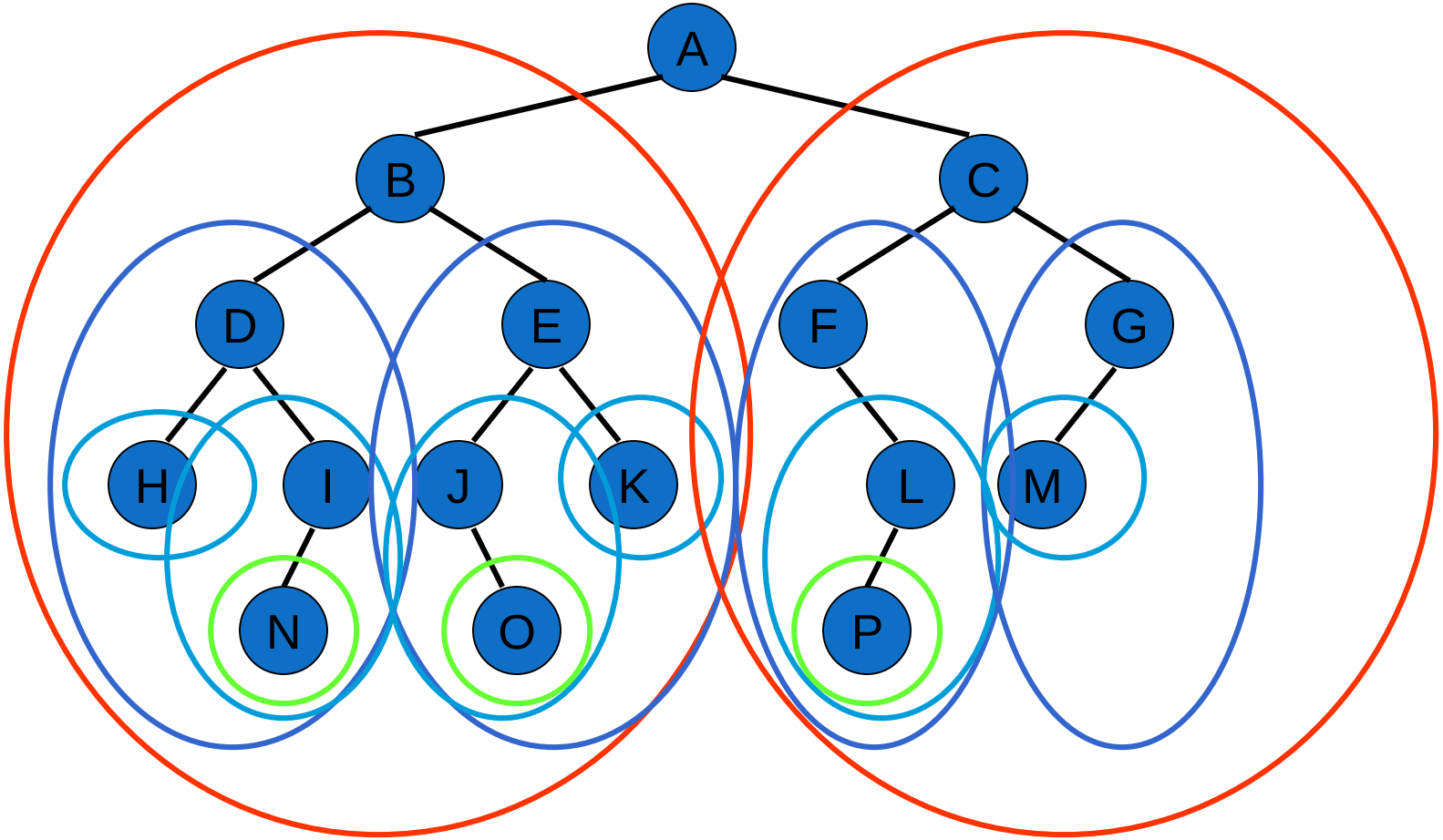


FIGURE 6-8 Binary Tree Traversals

- **Duyệt theo thứ tự trước** (Gốc-Trái-Phải)



Kết quả: A B D H I N E J O K C F L P G M

Retrieved from http://www.cse.hcmut.edu.vn/~ttqnguyet/Slides/CSDL/5_DataStructures.ppt

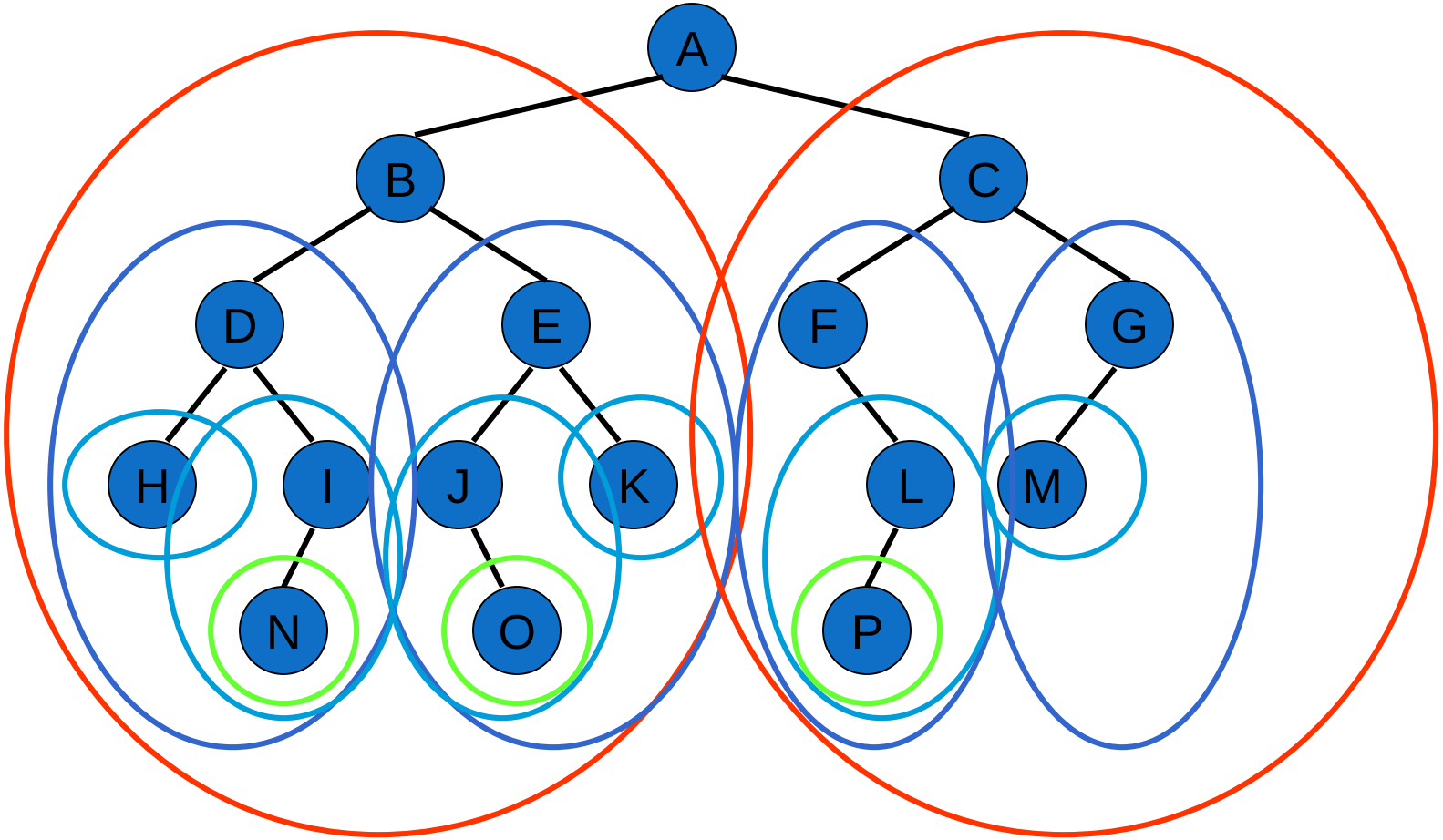
Duyệt cây nhị phân

- Duyệt theo thứ tự trước NLR (Node-Left-Right)
  - Kiểu duyệt này trước tiên thăm nút gốc sau đó thăm các nút của cây con trái rồi đến cây con phải
  - Thủ tục duyệt có thể trình bày đơn giản như sau:

```
void  NLR (Tree t)
{
      if (t != NULL)
      {
            // Xử lý t tương ứng theo nhu cầu
            NLR(t->pLeft);
            NLR(t->pRight);
      }
}
```

DUYỆT CÂY NHỊ PHÂN

- **DUYỆT THEO THỨ TỰ GIỮA (TRÁI-GỐC-PHẢI)**



Kết quả: H D N I B J O E K A F P L C M G

Retrieved from http://www.cse.hcmut.edu.vn/~ttqnguyet/Slides/CSDL/5_DataStructures.ppt

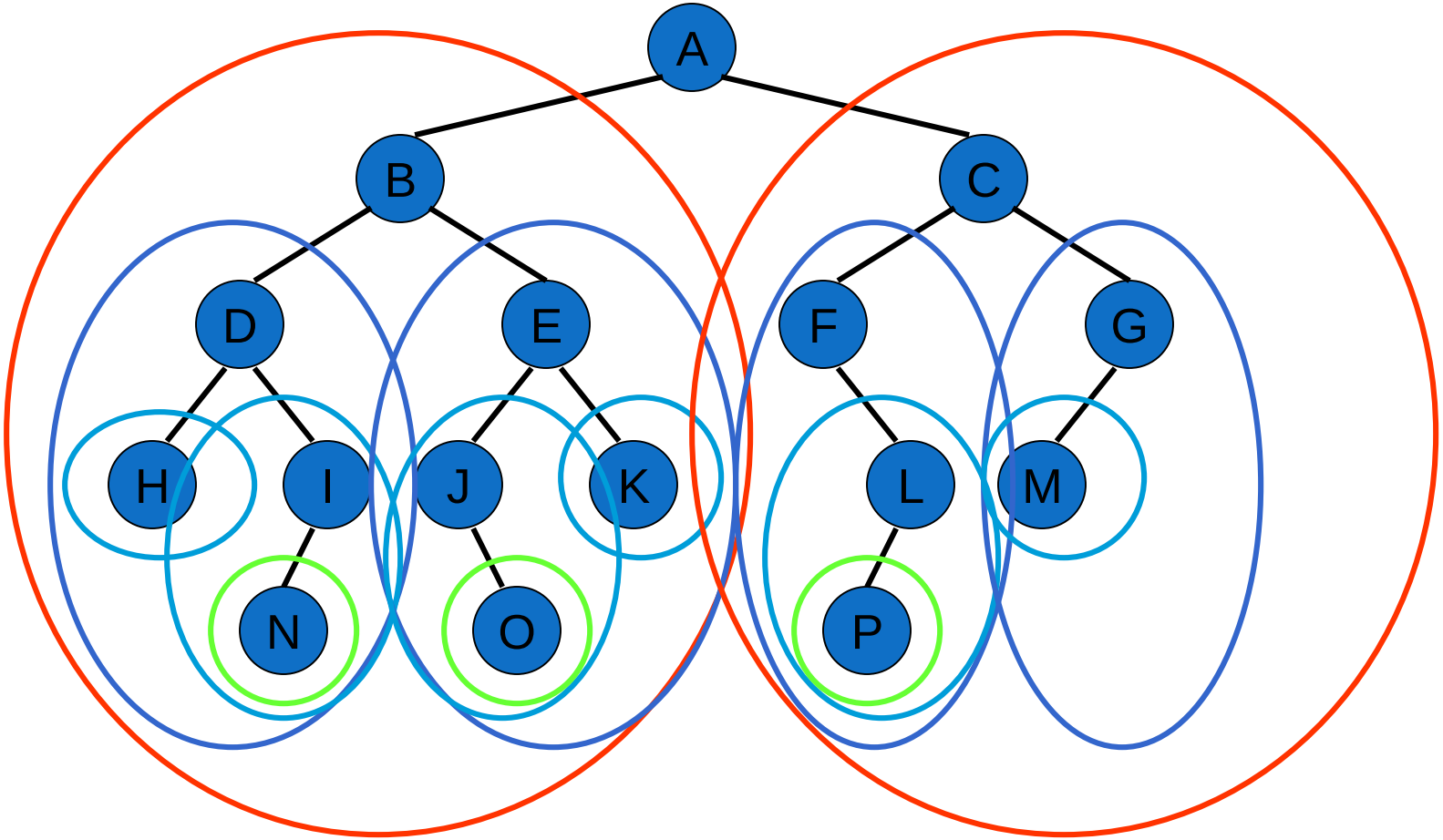
Duyệt cây nhị phân

- Duyệt theo thứ tự giữa LNR (Left-Node-Right)
  - Kiểu duyệt này trước tiên thăm các nút của cây con trái sau đó thăm nút gốc rồi đến cây con phải
  - Thủ tục duyệt có thể trình bày đơn giản như sau:

```
void LNR(Tree t)
{
      if (t != NULL)
      {
            LNR(t->pLeft);
            //Xử lý nút t theo nhu cầu
            LNR(t->pRight);
      }
}
```

DUYỆT CÂY NHỊ PHÂN

DUYỆT THEO THỨ TỰ SAU (TRÁI-PHẢI-GỐC)



Kết quả: H N I D O J K E B P L F M G C A

Retrieved from http://www.cse.hcmut.edu.vn/~ttqnguyet/Slides/CSDL/5_DataStructures.ppt

Duyệt cây nhị phân

- **Duyệt theo thứ tự sau (Trái-Phải-Gốc)**

- Kiểu duyệt này trước tiên thăm các nút của cây con trái sau đó thăm đến cây con phải rồi cuối cùng mới thăm nút gốc
- Thủ tục duyệt có thể trình bày đơn giản như sau:

```
void  LRN(Tree t)
{
      if (t != NULL)
      {
            LRN(t->pLeft);
            LRN(t->pRight);
            // Xử lý tương ứng t theo nhu cầu
      }
}
```

Cây nhị phân tìm kiếm

- **Cây nhị phân tìm kiếm (BST):** là một cây nhị phân rỗng hoặc mỗi nút của cây này có khóa lớn hơn khóa của tất cả các nút của cây con bên trái và bé hơn (hoặc bằng) khóa của tất cả các nút của cây con bên phải

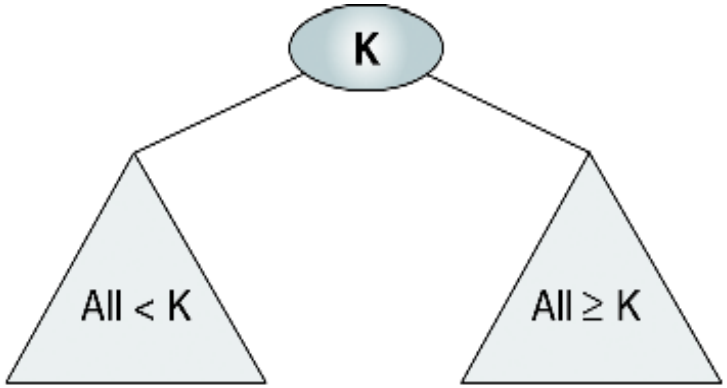


FIGURE 7-1 Binary Search Tree

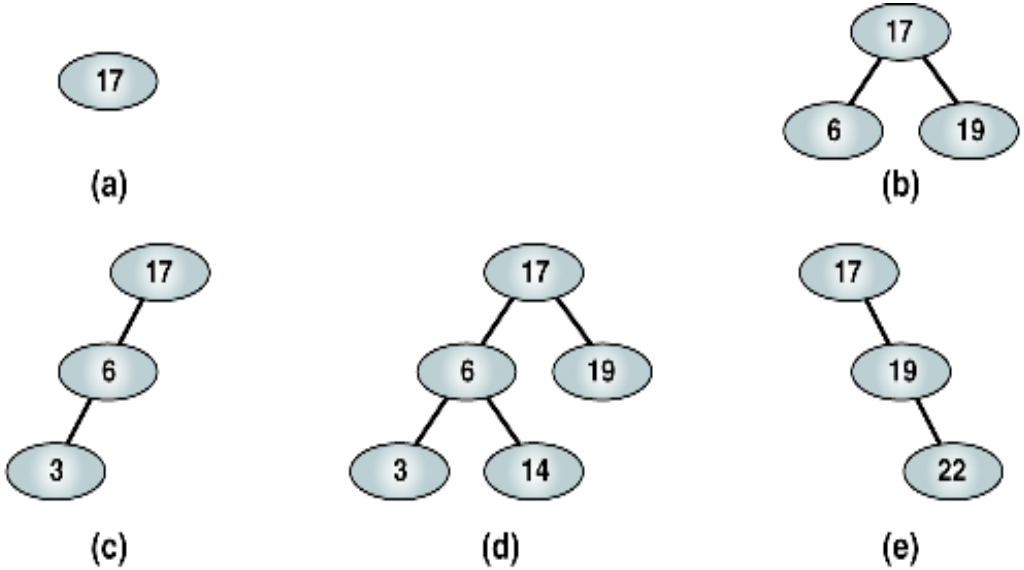


FIGURE 7-2 Valid Binary Search Trees

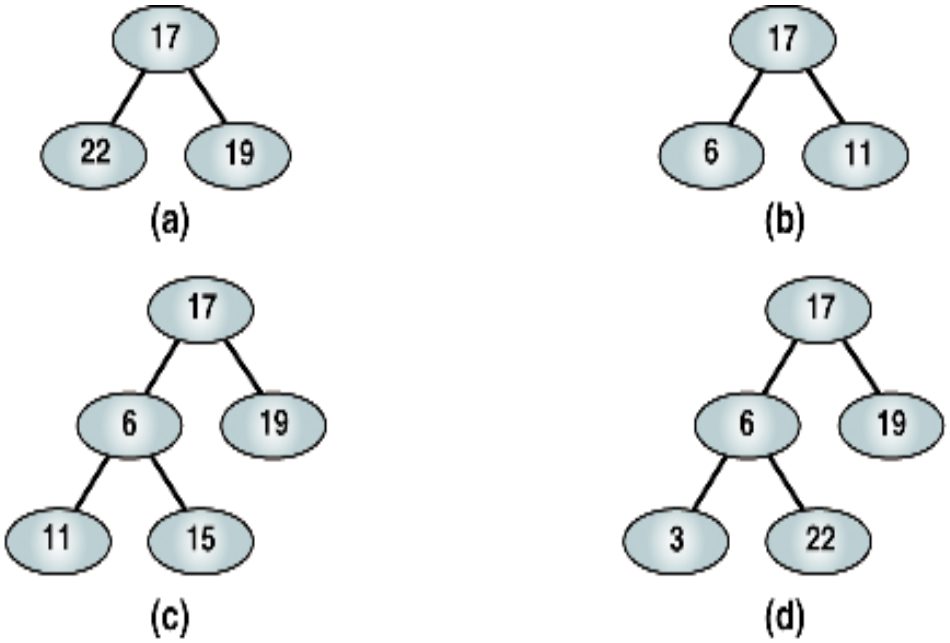
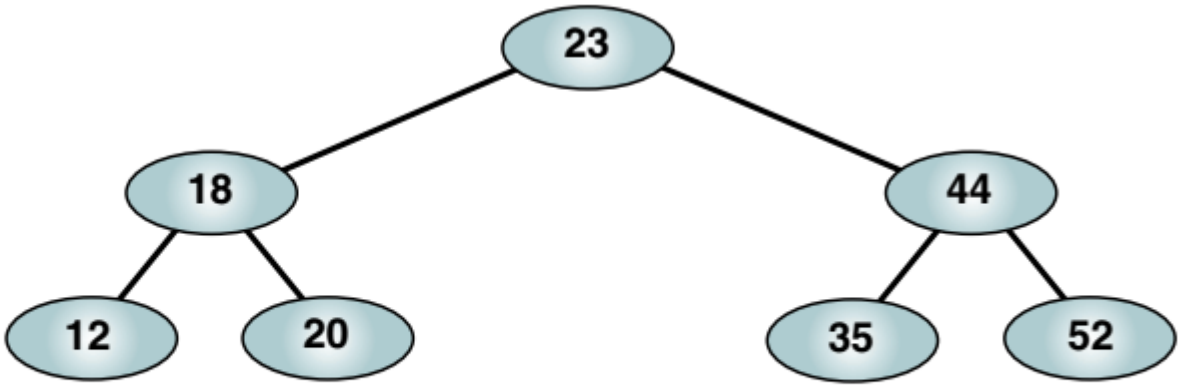


FIGURE 7-3 Invalid Binary Search Trees

DUYỆT CÂY NHỊ PHÂN TÌM KIẾM



Example of a Binary Search Tree

- Thứ tự trước: 23 18 12 20 44 35 52
- Thứ tự sau: 12 20 18 35 52 44 23
- Thứ tự giữa: **12 18 20 23 35 44 52**

Tìm kiếm trên BST

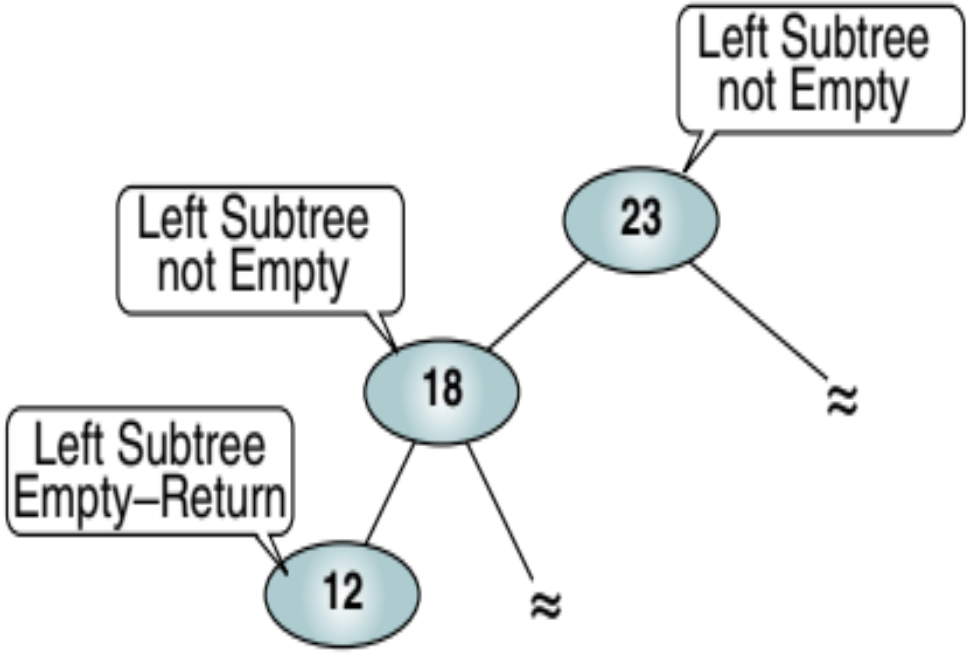
- Tìm nút nhỏ nhất
- Tìm nút lớn nhất
- Tìm nút bất kỳ

Tìm nút nhỏ nhất

- Nút có giá trị bé nhất là nút nằm tận cùng bên trái của cây.

Find Smallest Node in a BST

```
Algorithm findSmallestBST (root)
This algorithm finds the smallest node in a BST.
   Pre    root is a pointer to a nonempty BST or subtree
   Return address of smallest node
 1 if (left subtree empty)
   1   return (root)
 2 end if
 3 return findSmallestBST (left subtree)
end findSmallestBST
```



Find Smallest Node in a BST

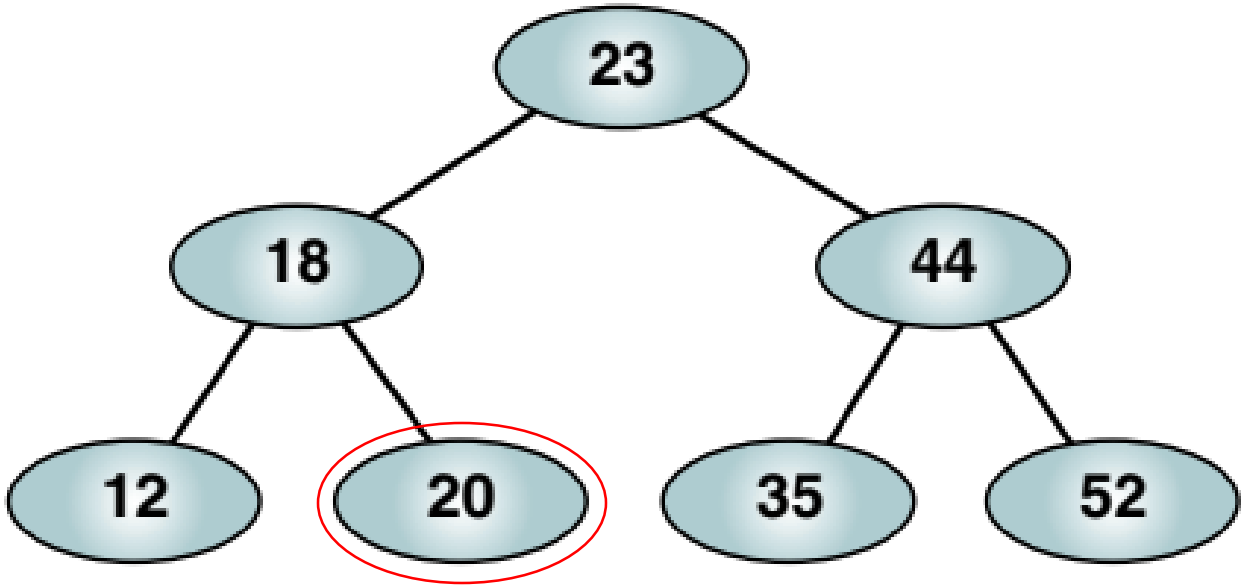
Tìm nút lớn nhất

- Bắt đầu từ gốc
- Di chuyển về phía nhánh phải tới nút xa nhất bên phải của cây

Find Largest Node in a BST

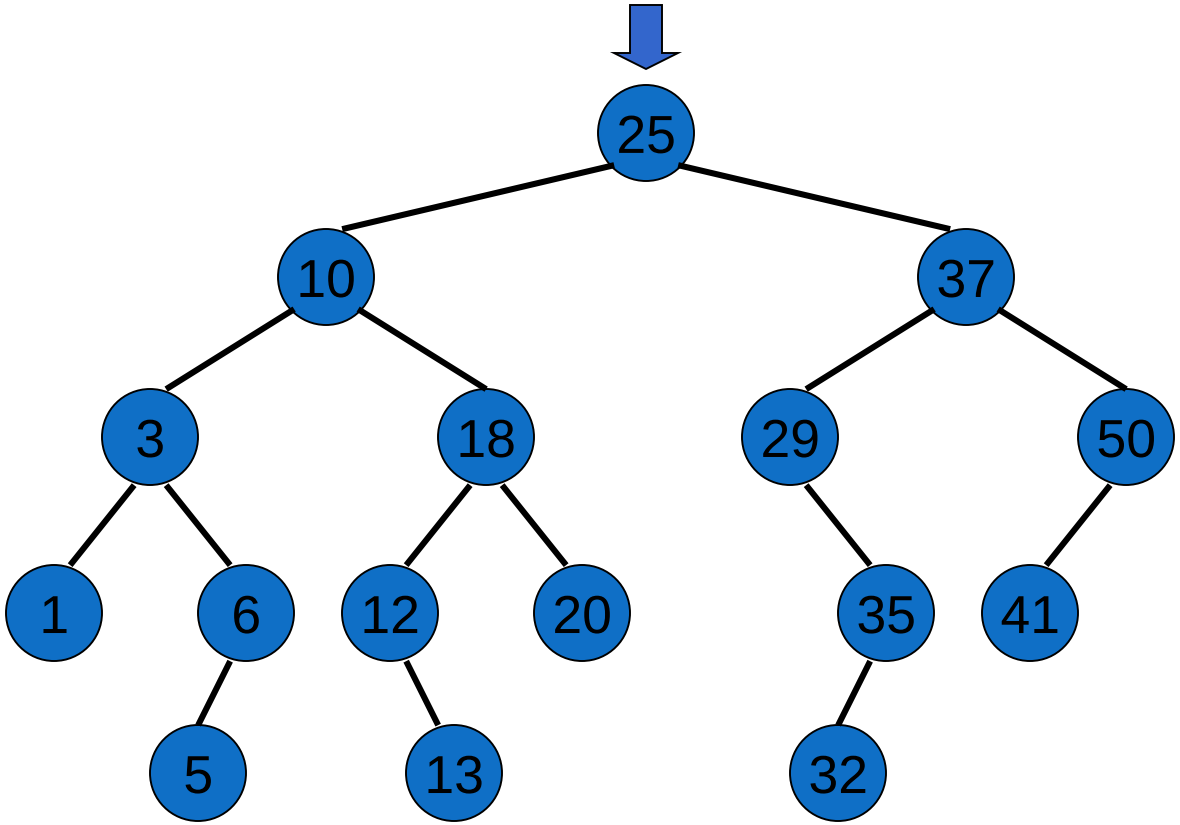
```
Algorithm findLargestBST (root)
This algorithm finds the largest node in a BST.
   Pre    root is a pointer to a nonempty BST or subtree
   Return address of largest node returned
 1 if (right subtree empty)
   1   return (root)
 2 end if
 3 return findLargestBST (right subtree)
end findLargestBST
```

Tìm nút bất kỳ trên BST



Search points in binary search

VÍ DỤ TÌM KIẾM TRÊN BST



Khóa gốc lớn hơn

Khóa gốc nhỏ hơn

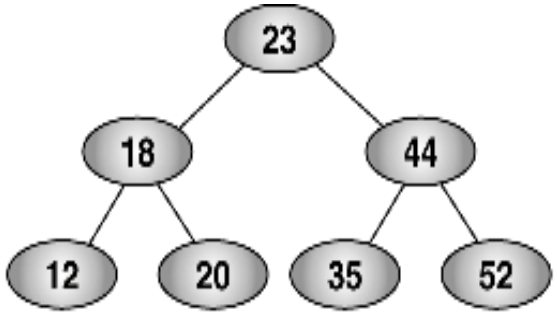
Tìm kiếm 13

Tìm thấy

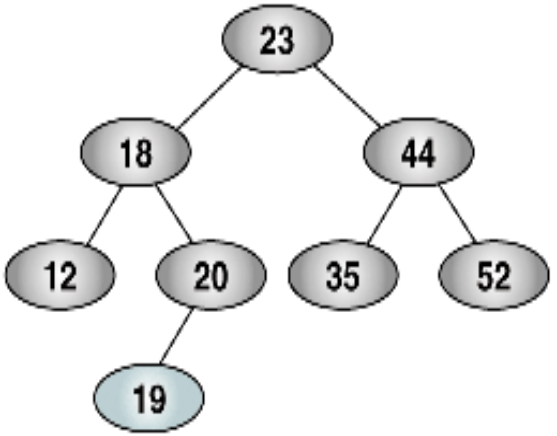
Search BST

```
Algorithm searchBST (root, targetKey)
Search a binary search tree for a given value.
   Pre    root is the root to a binary tree or subtree
          targetKey is the key value requested
   Return the node address if the value is found
          null if the node is not in the tree
 1  if (empty tree)
       Not found
    1  return null
 2  end if
 3  if (targetKey < root)
    1  return searchBST (left subtree, targetKey)
 4  else if (targetKey > root)
    1  return searchBST (right subtree, targetKey)
 5  else
       Found target key
    1  return root
 6  end if
end searchBST
```

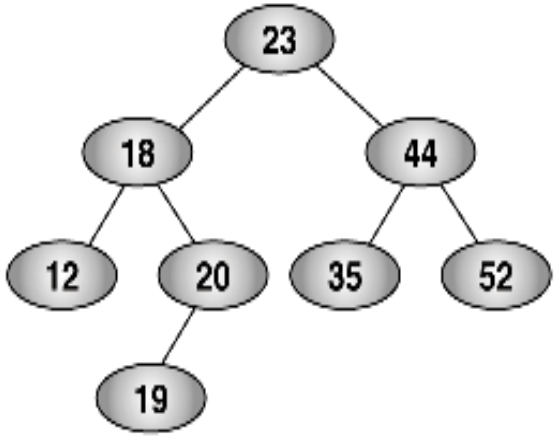
Thêm phần tử vào BST



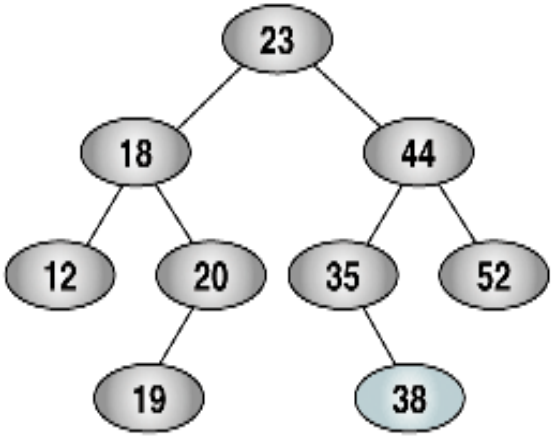
(a) Before inserting 19



(b) After inserting 19



(c) Before inserting 38



(d) After inserting 38

Add Node to BST

```
Algorithm addBST (root, newNode)
Insert node containing new data into BST using recursion.
   Pre    root is address of current node in a BST
          newNode is address of node containing data
   Post   newNode inserted into the tree
   Return address of potential new tree root
1  if (empty tree)
   1   set root to newNode
   2   return newNode
2  end if

   Locate null subtree for insertion
3  if (newNode < root)
   1   return addBST (left subtree, newNode)
4  else
   1   return addBST (right subtree, newNode)
5  end if
end addBST
```

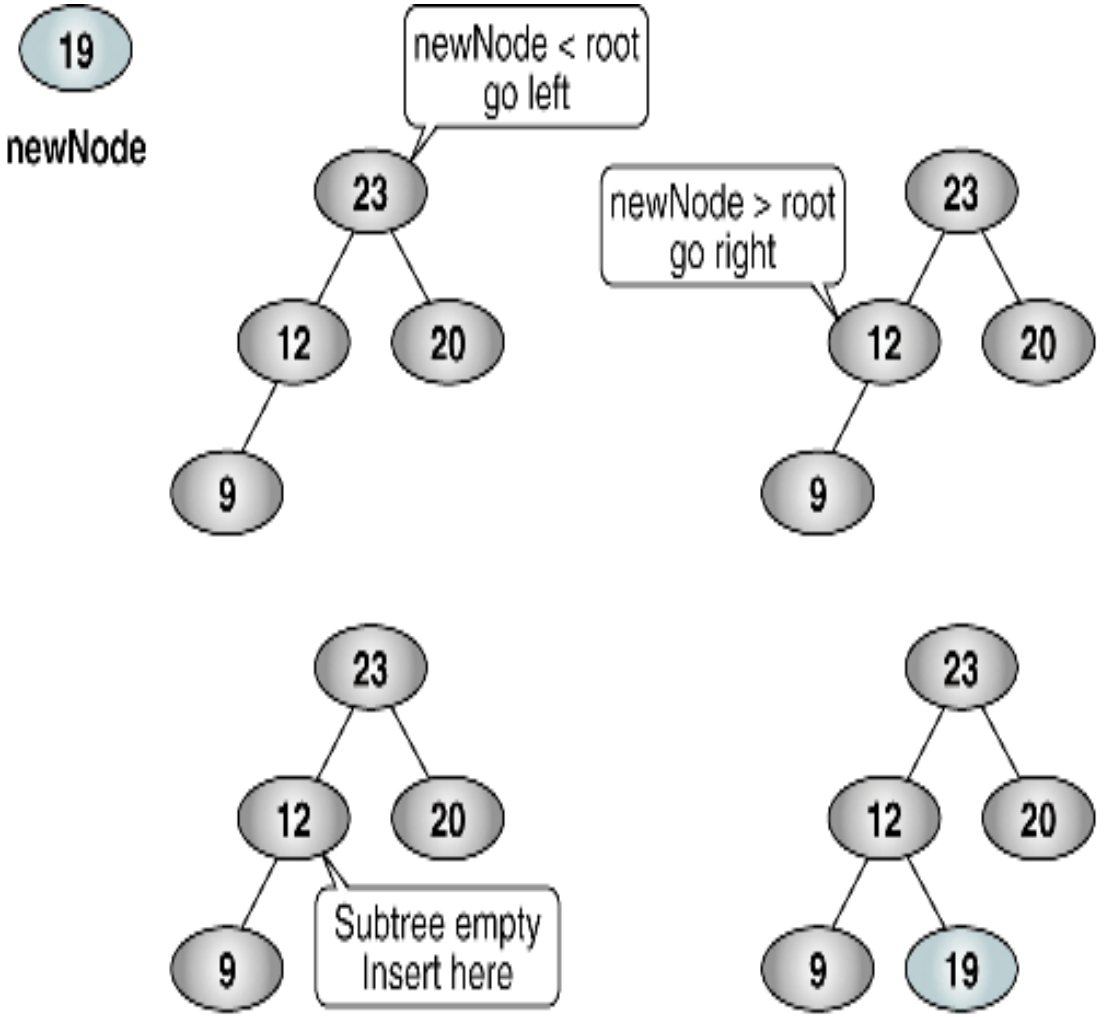
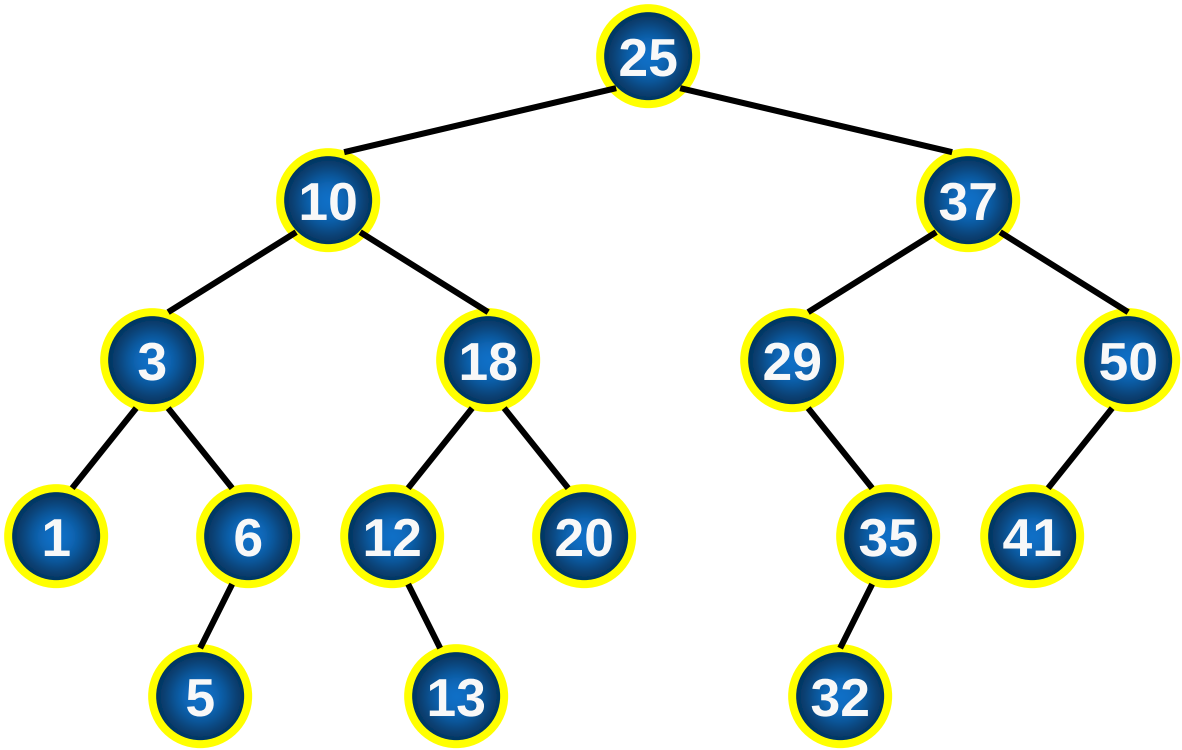


FIGURE 7-9 Trace of Recursive BST Insert

TẠO CÂY NHỊ PHÂN TÌM KIẾM

25 37 10 18 29 50 3 1 6 5 12 20 35 13 32 41



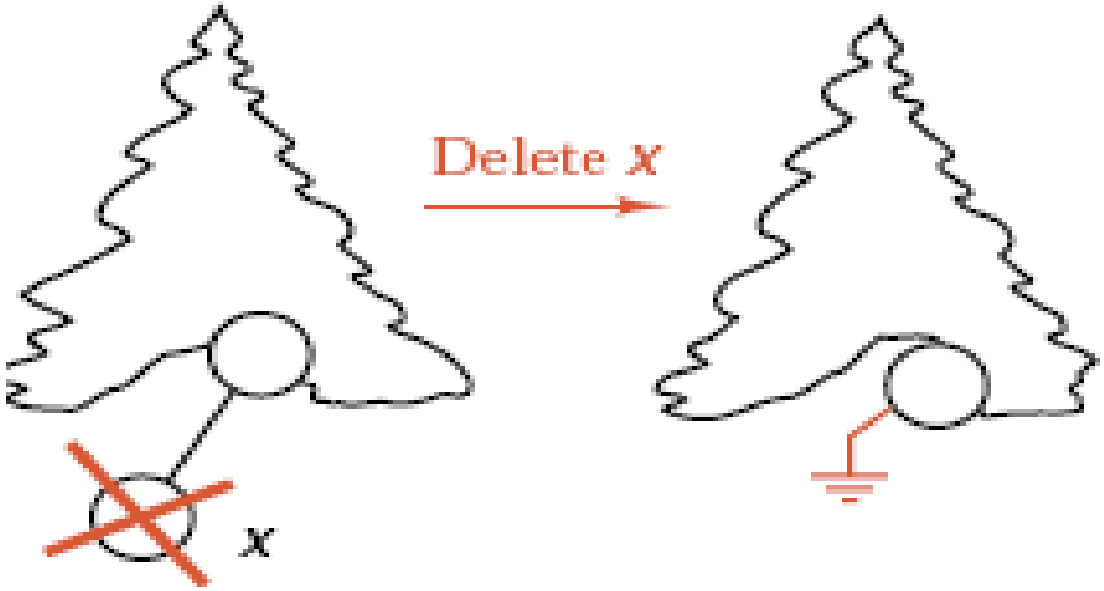
25	37	10	18	29	50	3	1	6	5	12	20	35	13	32	41

Loại bỏ phần tử trên BST

- **Việc loại bỏ một phần tử X ra khỏi cây phải bảo đảm điều kiện ràng buộc của CNPTK.**
- **Có 4 trường hợp khi loại bỏ nút X có thể xảy ra:**
  - X là nút lá
  - X chỉ có 1 con trái
  - X chỉ có 1 con phải.
  - X có đủ cả 2 con

Loại bỏ phần tử trên BST

- Trường hợp 1: X là nút lá.

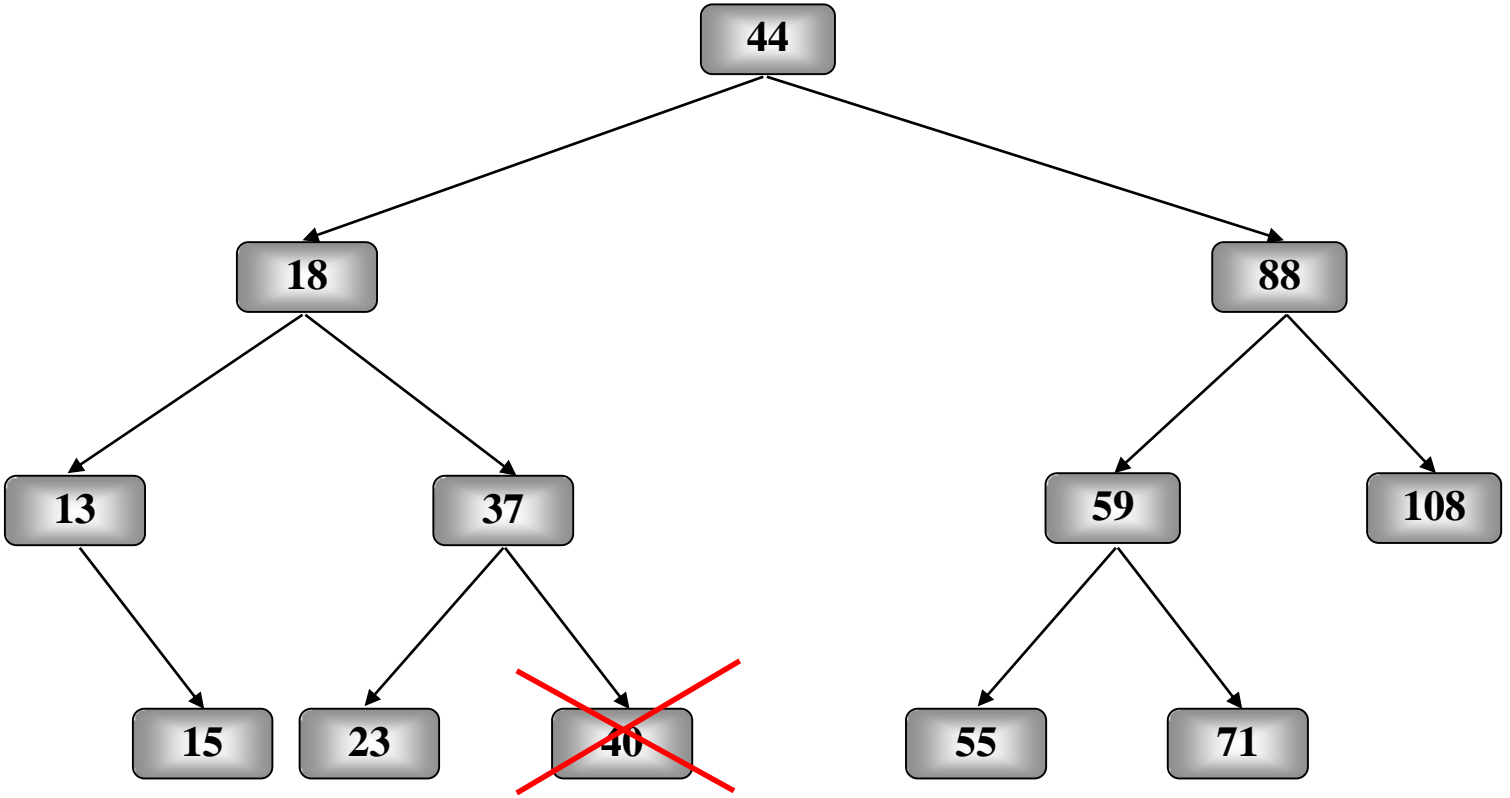


1. **Xóa nút này**

2. **Gán liên kết từ nút cha của nó thành rỗng**

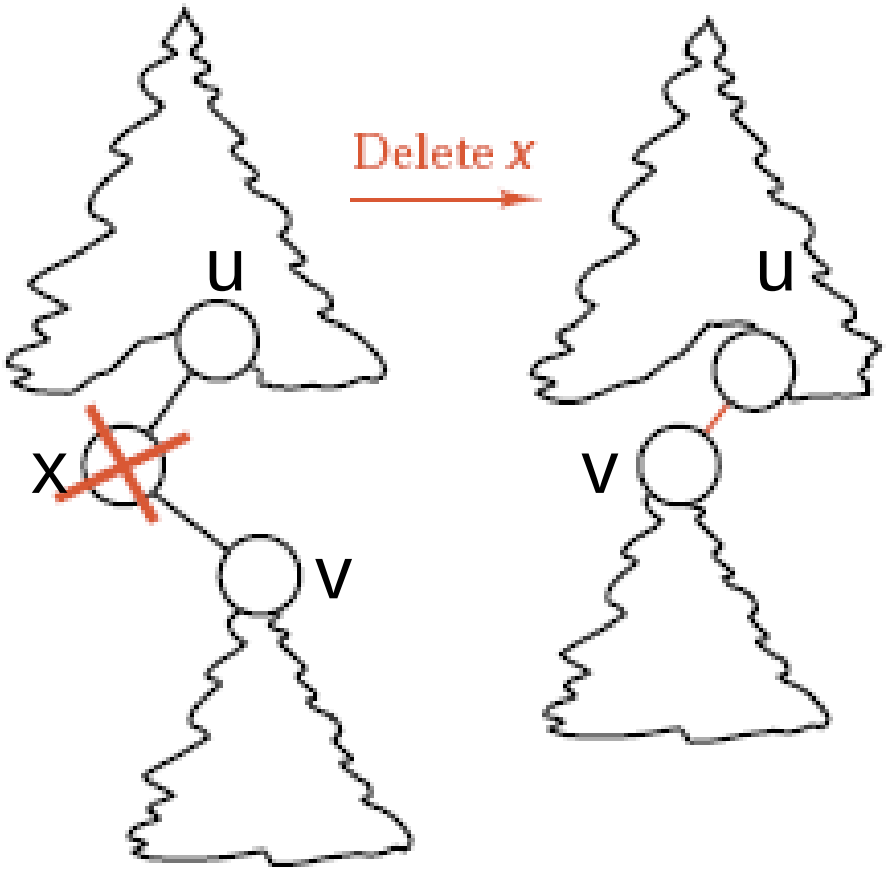
Loại bỏ phần tử trên BST

- Trường hợp 1: X là nút lá.



Loại bỏ phần tử trên BST

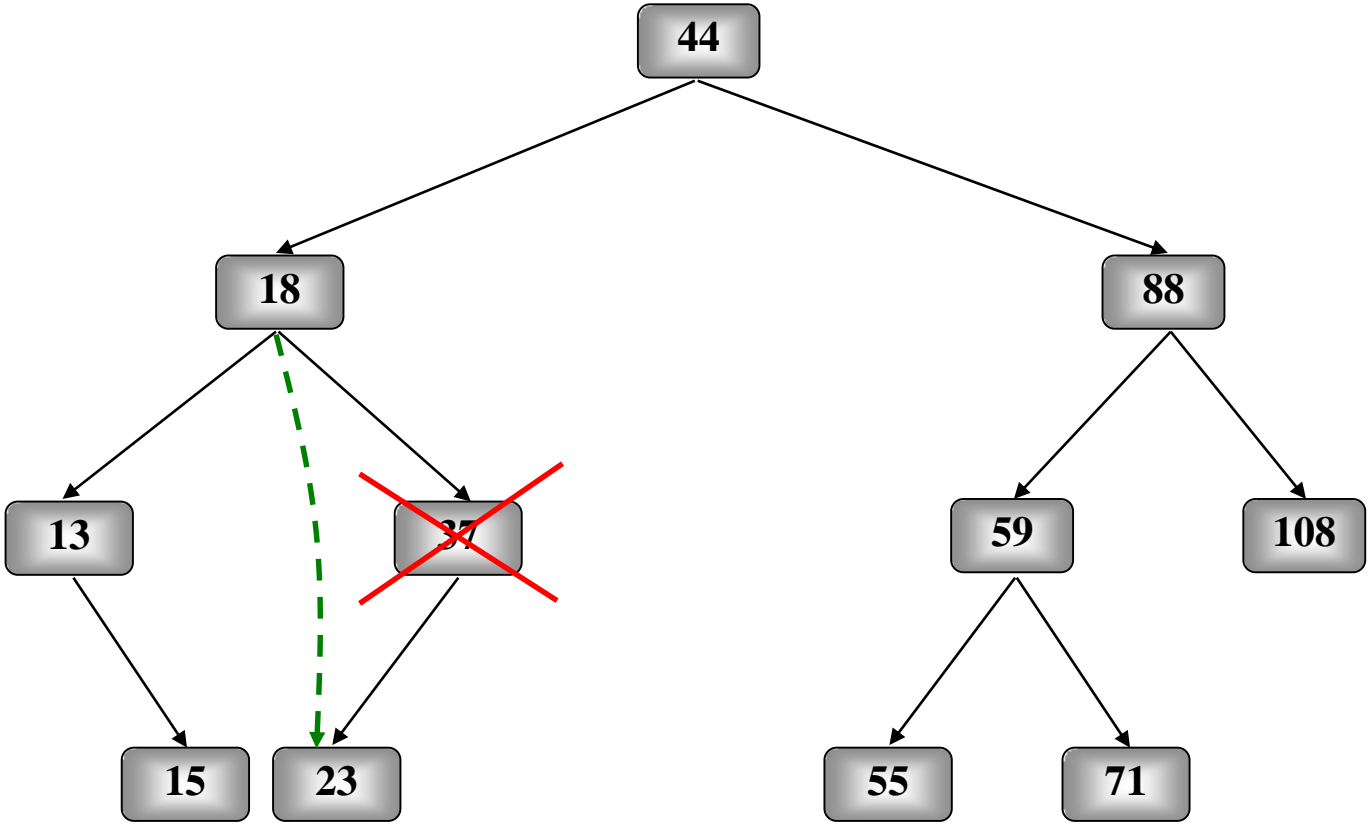
- Trường hợp 2,3: X chỉ có 1 con (trái hoặc phải)



1. **Gán liên kết từ cha của nó xuống con duy nhất của nó**
2. **Xóa nút này**

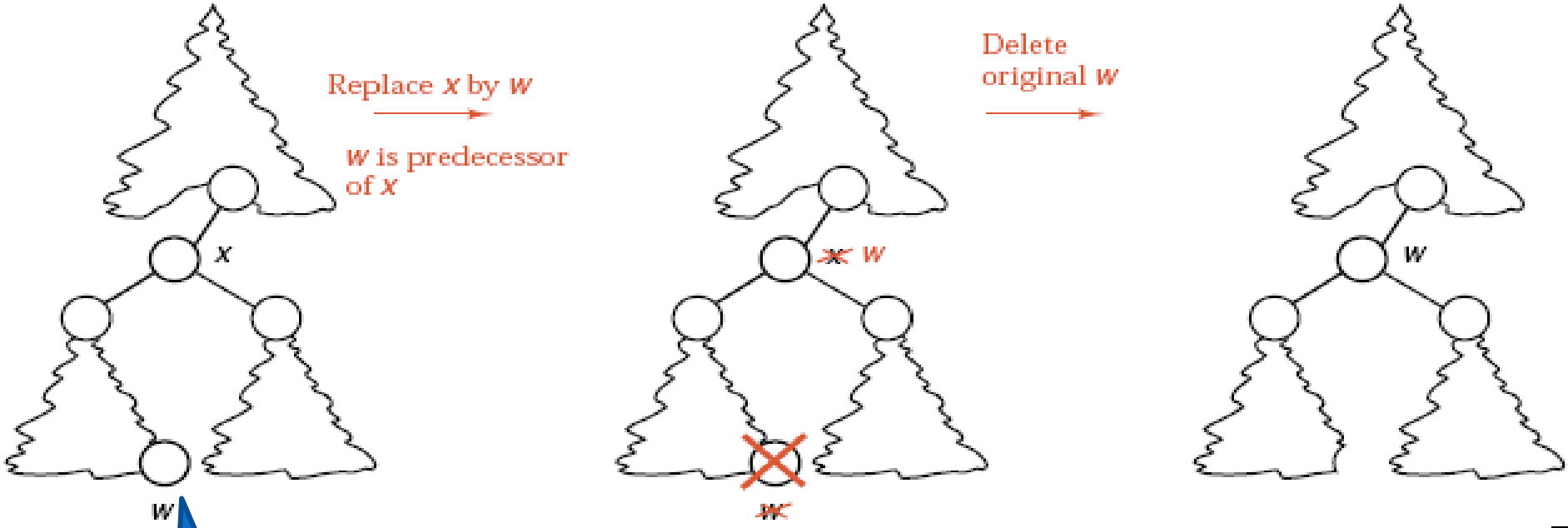
Loại bỏ phần tử trên BST

- Trường hợp 2: X chỉ có 1 con (trái hoặc phải)



Loại bỏ phần tử trên BST

- Trường hợp 4: X có đủ 2 con



Nút cực **phải** của cây con **trái** của x

1. Tìm w là nút cực phải (lớn nhất) của cây con trái của X (hoặc cực trái (bé nhất) của cây con phải của X)

2. Thay x bằng w

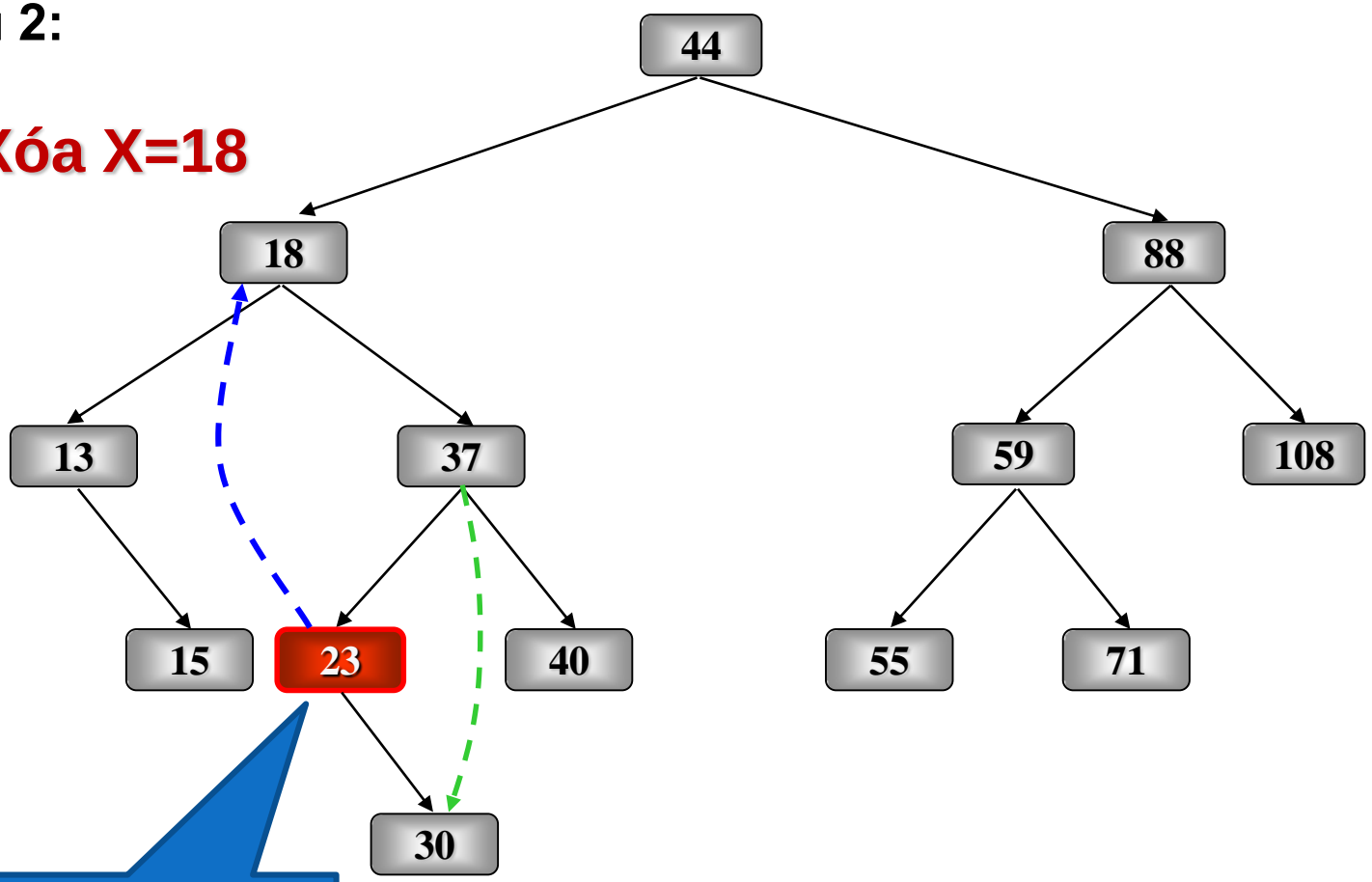
3. Xóa nút w cũ (giống trường hợp 1 hoặc 2 đã xét)

Loại bỏ phần tử trên BST

- Trường hợp 4: X có đủ 2 con

Ví dụ 2:

Xóa X=18



LOẠI BỎ PHẦN TỬ TRÊN BST

```
Algorithm deleteBST (root, dltKey)
   Return true if node deleted, false if not found
1  if (empty tree)
   1   return false
2  end if
3  if (dltKey < root)
   1   return deleteBST (left subtree, dltKey)
4  else if (dltKey > root)
   1   return deleteBST (right subtree, dltKey)
5  else
       Delete node found--test for leaf node
   1   If (no left subtree) // if ( root -> left = null)  (root không có cây con trái)
       1   make right subtree the root   // root = root->right
       2   return true
   2   else if (no right subtree) // if (root -> right = null)  (root không có cây con phải)
       1   make left subtree the root // root = root->left
       2   return true
   3   else
          Node to be deleted not a leaf. Find largest node on
          left subtree.
       1   save root in deleteNode   // deleteNode = root
       2   set largest to largestBST (left subtree) // largest = largestBTS(root->left)
       3   move data in largest to deleteNode // deleteNode ->data = largest ->data
       4   return deleteBST (left subtree of deleteNode,
                             key of largest  // return deleteBST(deleteNode->left,
                                             largest ->key) (key là giá trị khóa)
   4   end if
6  end if
end deleteBST
```